CSCE 775: Reinforcement Learning
Quiz 1 Summary
Chapters 1, 3, and 4

J.C. Vaught

January 28, 2026

Contents

1 Reinforcement Learning Fundamentals

Reinforcement learning (RL) is a computational approach to understanding and automating goal-directed learning from interaction. The learner, called an *agent*, learns how to map situations to actions so as to maximize a numerical reward signal over time. Unlike supervised learning, which learns from labeled examples provided by an external supervisor, RL must discover which actions yield the most reward through trial and error. The agent is not told which actions to take but instead must discover which actions yield the highest cumulative reward by trying them.

1.1 Distinguishing Features of Reinforcement Learning

Two key features distinguish RL from other computational approaches to learning. First, RL emphasizes *trial-and-error search*: the agent must actively explore different actions to discover their effects rather than being told the correct action. Second, RL involves *delayed reward*: actions may have consequences that play out over many time steps, requiring the agent to consider long-term effects rather than just immediate rewards.

These features position RL as a third paradigm in machine learning, distinct from supervised and unsupervised learning. While supervised learning learns from labeled examples and unsupervised learning finds hidden structure in unlabeled data, RL learns from evaluative feedback that indicates how good an action was but not whether it was the best possible action.

1.2 The Exploration-Exploitation Dilemma

A fundamental challenge in RL is the tension between exploration and exploitation. To maximize reward, the agent must exploit actions it has already discovered to be effective. However, to discover better actions, it must explore actions it has not tried before. The agent cannot exclusively exploit without potentially missing better alternatives, nor can it exclusively explore without sacrificing reward. Balancing exploration and exploitation is one of the central challenges that distinguishes RL from other learning paradigms.

1.3 The Four Elements of Reinforcement Learning

Beyond the agent and environment, four main subelements comprise a reinforcement learning system:

Policy. The policy defines the agent's behavior at a given time. Formally, it is a mapping from perceived states of the environment to actions to be taken. The policy may be a simple lookup table, a search process, or a complex function approximator. It is the core of an RL agent in the sense that it alone is sufficient to determine behavior.

Reward Signal. At each time step, the environment sends the agent a single number called the reward. The agent's sole objective is to maximize the total reward over time. The reward signal defines what are the good and bad events for the agent and is the primary basis for altering the policy. If an action selected by the policy is followed by low reward, the policy may change to select a different action in that situation in the future.

Value Function. While the reward signal indicates immediate desirability, the value function specifies long-term desirability. The value of a state is the total amount of reward an agent can expect to accumulate starting from that state. Values indicate the long-term desirability of states after considering the states that are likely to follow and the rewards available in those states. Actions are chosen based on value judgments: we seek actions that bring about states of highest value, not highest reward, because these actions obtain the greatest reward over the long run.

Model (Optional). Some RL systems include a model of the environment, which mimics the behavior of the environment and allows inferences to be made about how the environment will behave. Given a state and action, the model might predict the resultant next state and reward. Models are used for planning: deciding on a course of action by considering possible future situations before they are actually experienced.

1.4 Model-Based vs. Model-Free Methods

Methods that use models are called *model-based* methods, while those that rely purely on trial-and-error are *model-free* methods. Model-free methods are explicitly trial-and-error learners that view the RL problem as one of finding a good policy or value function through direct interaction. Modern RL spans the spectrum from low-level, trial-and-error learning to high-level, deliberative planning.

1.5 The Tic-Tac-Toe Example

The tic-tac-toe example illustrates a value function approach to RL. We create a table with one entry for each possible state of the game, representing our estimate of the probability of winning from that state. This table defines our current value function. Most of the time, we select moves greedily by choosing the move that leads to the state with the highest value (exploitation). Occasionally, however, we select randomly from among the available moves (exploration).

As we play, we update our value estimates using a *temporal-difference* learning rule. After each greedy move, we update the value of the previous state to be closer to the value of the current state:

$$V(S_t) \leftarrow V(S_t) + \alpha[V(S_{t+1}) - V(S_t)] \tag{1}$$

where S_t denotes the state before the move, S_{t+1} the state after the move, and α is a step-size parameter that controls the rate of learning. This update rule drives the value of the earlier state toward the value of the later state, a principle called temporal-difference learning.

1.6 Historical Threads

Three main threads of research have contributed to modern RL. The first is the idea of *trial-and-error learning*, which has origins in psychology and the study of animal learning. The second is the problem of *optimal control* and its solution using dynamic programming, which provides the mathematical foundation for much of RL. The third is *temporal-difference methods*, which combine the sampling of trial-and-error methods with the bootstrapping of dynamic programming.

2 The Agent-Environment Interface

The learner and decision maker is called the *agent*. The thing it interacts with, comprising everything outside the agent, is called the *environment*. These interact continually: the agent selects actions and the environment responds with new situations and rewards.

2.1 The MDP Framework

More precisely, the agent and environment interact at each of a sequence of discrete time steps $t = 0, 1, 2, 3, \ldots$ At each time step t, the agent receives some representation of the environment's state $S_t \in \mathcal{S}$, and on that basis selects an action $A_t \in \mathcal{A}(s)$. One time step later, in part as a consequence of its action, the agent receives a numerical reward $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$ and finds itself in a new state S_{t+1}.

This produces a trajectory or sequence:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \ldots \tag{2}$$

In a finite Markov decision process (MDP), the sets of states, actions, and rewards all have a finite number of elements. The random variables R_t and S_t have well-defined discrete probability distributions dependent only on the preceding state and action.

2.2 The Dynamics Function

The dynamics function arises directly from the Markov property. Because the current state S_{t-1} captures all relevant information about the past, the joint probability of the next state and reward depends only on the immediately preceding state and action—not on the full history. This allows us to express the environment's behavior as a single conditional probability distribution:

$$p(s', r \mid s, a) \doteq \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\} \tag{3}$$

for all $s', s \in \mathcal{S}$, $r \in \mathcal{R}$, and $a \in \mathcal{A}(s)$. This is a four-argument function $p : \mathcal{S} \times \mathcal{R} \times \mathcal{S} \times \mathcal{A} \to [0, 1]$.

The dynamics function is a *joint* conditional probability over both the next state s' and the reward r, given the current state s and action a. It is a proper probability distribution, so it must satisfy:

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) = 1, \quad \text{for all } s \in \mathcal{S},\ a \in \mathcal{A}(s) \tag{4}$$

This single function completely specifies the environment. Every other quantity of interest—state-transition probabilities, expected rewards, value functions, optimal policies—can be derived from it. In particular, we can marginalize or condition p to obtain three commonly used derived quantities:

State-transition probabilities (marginalize out r):

$$p(s' \mid s, a) = \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \tag{5}$$

This gives the probability of transitioning to state s' when taking action a in state s, regardless of what reward is received.

Expected reward for a state-action pair (expected value of r):

$$r(s,a) = \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r \mid s, a) \tag{6}$$

This is the expected immediate reward when taking action a in state s.

Expected reward for a state-action-next-state triple (condition on s', then take expectation over r):

$$r(s,a,s') = \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a, S_t = s'] = \sum_{r \in \mathcal{R}} r \frac{p(s', r \mid s, a)}{p(s' \mid s, a)} \tag{7}$$

Here the fraction $\frac{p(s',r|s,a)}{p(s'|s,a)}$ is just the conditional probability $p(r \mid s, a, s')$ by Bayes' rule, so this is the expected reward given that the specific transition $s \to s'$ occurs under action a.

2.3 The Markov Property

The state must include information about all aspects of the past agent-environment interaction that make a difference for the future. A state signal that succeeds in retaining all relevant information is said to be *Markov*, or to have the Markov property. Formally:

$$\Pr\{S_t = s', R_t = r \mid S_0, A_0, R_1, \ldots, S_{t-1}, A_{t-1}\} = \Pr\{S_t = s', R_t = r \mid S_{t-1}, A_{t-1}\} \tag{8}$$

The Markov property is important because it enables the agent to predict the next state and reward given only the current state and action, without requiring the complete history of past interactions.

2.4 The Agent-Environment Boundary

The boundary between agent and environment is not the same as the physical boundary of the robot or organism. The general rule is that anything that cannot be changed arbitrarily by the agent is considered part of the environment. The agent-environment boundary represents the limit of the agent's *absolute control*, not of its knowledge. For example, the reward computation is considered part of the environment even if it is physically inside the agent, because the agent cannot change it arbitrarily.

3 Goals, Rewards, and Returns

3.1 The Reward Hypothesis

The *reward hypothesis* states that all of what we mean by goals and purposes can be well thought of as the maximization of the expected value of the cumulative sum of a received scalar signal (called reward). This is a bold claim, but it provides a simple and mathematically tractable framework for formalizing goals.

The reward signal must capture *what* we want accomplished, not *how* we want it accomplished. For example, in chess, the reward should be based on winning and losing, not on achieving subgoals like taking the opponent's pieces or gaining control of the center. The agent must be free to discover for itself how best to achieve the goal.

3.2 Episodic and Continuing Tasks

Agent-environment interactions naturally break into subsequences called *episodes* in many problems. Each episode ends in a special state called the *terminal state*, followed by a reset to a standard starting state. Tasks with episodes are called *episodic tasks*. In episodic tasks, we sometimes need to distinguish the set of all nonterminal states, denoted $\mathcal{S}$, from the set of all states including terminal states, denoted $\mathcal{S}^+$.

In contrast, many tasks go on continually without limit. These are called *continuing tasks*. The distinction between episodic and continuing tasks requires different formal treatments of the return.

3.3 The Return and Discounting

The agent's goal is to maximize the expected *return*, where the return G_t is defined as some specific function of the reward sequence. In the simplest case, the return is the sum of future rewards:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \cdots + R_T \tag{9}$$

where T is the final time step. This makes sense for episodic tasks with a natural terminal time step.

For continuing tasks, the return could easily be infinite, making it problematic to maximize. The solution is to use *discounting*. The agent tries to maximize the *discounted return*:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{10}$$

where γ is a parameter, $0 \leq \gamma \leq 1$, called the *discount factor*.

The discount factor determines the present value of future rewards. A reward received k time steps in the future is worth only γ^{k-1} times what it would be worth if received immediately. When $\gamma = 0$, the agent is myopic and concerned only with maximizing immediate reward. As γ approaches 1, the agent becomes more farsighted and takes future rewards into account more strongly.

3.4 Key Properties of Returns

The return has an important recursive relationship:

$$G_t = R_{t+1} + \gamma G_{t+1} \tag{11}$$

This relationship holds even if the horizon is infinite or the episode terminates at T. For the terminal state, we define $G_T = 0$.

For the special case of a constant reward of +1 at every time step in a continuing task:

$$G_t = \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1-\gamma} \tag{12}$$

3.5 Unified Notation

We can unify episodic and continuing tasks by treating episodes as if they go on forever, with the terminal state transitioning only to itself with a reward of zero. This *absorbing state* trick allows us to write the return as:

$$G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k \tag{13}$$

where $T = \infty$ or $\gamma = 1$ (but not both) is allowed. This convention simplifies notation throughout the rest of the theory.

4 Policies and Value Functions

Almost all RL algorithms involve estimating *value functions*: functions of states (or of state-action pairs) that estimate how good it is for the agent to be in a given state (or to perform a given action in a given state). The notion of "how good" is defined in terms of expected future rewards.

4.1 Policies

A *policy* is a mapping from states to probabilities of selecting each possible action. If the agent is following policy π at time t, then $\pi(a \mid s)$ is the probability that $A_t = a$ if $S_t = s$. Reinforcement learning methods specify how the agent's policy is changed as a result of its experience.

4.2 The State-Value Function

The *value function* of a state s under policy π, denoted $v_\pi(s)$, is the expected return when starting in s and following π thereafter:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right] \tag{14}$$

for all $s \in \mathcal{S}$. We call v_π the *state-value function for policy* π.

4.3 The Action-Value Function

Similarly, we define the value of taking action a in state s under policy π, denoted $q_\pi(s, a)$, as the expected return starting from s, taking action a, and thereafter following π:

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right] \tag{15}$$

We call q_π the *action-value function for policy* π.

4.4 Relationship Between Value Functions

The state-value and action-value functions are related:

$$v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) q_\pi(s, a) \tag{16}$$

This equation states that the value of a state under π is the expected value over actions selected according to π of the action values for those actions.

4.5 The Bellman Equation for v_π

A fundamental property of value functions is that they satisfy recursive relationships. The Bellman equation for v_π expresses the value of a state in terms of the values of successor states:

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a)[r + \gamma v_\pi(s')] \tag{17}$$

for all $s \in \mathcal{S}$.

This equation is the foundation for many RL algorithms. It expresses a relationship between the value of a state and the values of its successor states. The value of the start state must equal the discounted value of the expected next state plus the reward expected along the way.

The Bellman equation averages over all possibilities, weighting each by its probability of occurring. It states that the value of the start state is the expected immediate reward plus the expected discounted value of the next state, assuming that we select actions according to policy π.

4.6 System of Linear Equations

The Bellman equation is actually a system of $|\mathcal{S}|$ linear equations in $|\mathcal{S}|$ unknowns (the values $v_\pi(s)$ for all $s \in \mathcal{S}$). If the dynamics p of the environment are known, then this is a straightforward system of linear equations that can be solved by any number of methods. When $\gamma < 1$ or when eventual termination is guaranteed from all states, this system has a unique solution.

4.7 Backup Diagrams

Backup diagrams provide a visual representation of the Bellman equations. They show the relationships between states and actions that form the basis of the update or backup operations that are the heart of RL methods. For the state-value function, the backup diagram shows a state node at the top, action nodes below it (weighted by $\pi(a \mid s)$), and state nodes at the bottom (weighted by $p(s' \mid s, a)$). The diagram illustrates how value information is "backed up" from successor states to the current state.

4.8 Gridworld Example

Consider a 5×5 gridworld with special states A and B. From state A, all actions yield a reward of $+10$ and take the agent to state A'. From state B, all actions yield a reward of $+5$ and take the agent to state B'. From all other states, each action yields a reward of 0, and actions that would take the agent off the grid leave the state unchanged with a reward of -1.

Under a random policy (each action equally probable), we can compute the value function v_π by iteratively applying the Bellman equation. The values converge to a unique solution that reflects the long-term desirability of each state under the random policy.

5 Optimal Policies and Optimal Value Functions

5.1 Partial Ordering of Policies

Value functions define a partial ordering over policies. A policy π is better than or equal to a policy π' if its expected return is greater than or equal to that of π' for all states:

$$\pi \geq \pi' \text{ if and only if } v_\pi(s) \geq v_{\pi'}(s) \text{ for all } s \in \mathcal{S} \tag{18}$$

5.2 Optimal Policies

There is always at least one policy that is better than or equal to all other policies. This is an *optimal policy*, denoted π_*. Optimal policies share the same state-value function, called the *optimal state-value function*, denoted v_*:

$$v_*(s) = \max_\pi v_\pi(s) \tag{19}$$

for all $s \in \mathcal{S}$.

Optimal policies also share the same *optimal action-value function*, denoted q_*:

$$q_*(s, a) = \max_\pi q_\pi(s, a) \tag{20}$$

for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$. For the state-action pair (s, a), this gives the expected return for taking action a in state s and thereafter following an optimal policy.

We can write q_* in terms of v_*:

$$q_*(s, a) = \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \tag{21}$$

5.3 Bellman Optimality Equation for v_*

Because v_* is the value function for a policy, it must satisfy the Bellman equation for state values. However, because it is the optimal value function, we can write it without reference to any specific policy:

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s,a) = \max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_*(s')] \tag{22}$$

This is the *Bellman optimality equation for* v_*. It expresses the fact that the value of a state under an optimal policy must equal the expected return for the best action from that state.

5.4 Bellman Optimality Equation for q_*

Similarly, the Bellman optimality equation for q_* is:

$$q_*(s,a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right] \tag{23}$$

5.5 Extracting Optimal Policies

Once we have v_*, it is relatively easy to determine an optimal policy. For each state s, there will be one or more actions at which the maximum in the Bellman optimality equation is achieved. Any policy that assigns nonzero probability only to these actions is an optimal policy:

$$\pi_*(a \mid s) = \begin{cases} 1 & \text{if } a = \arg\max_{a'} \sum_{s',r} p(s', r \mid s, a')[r + \gamma v_*(s')] \\ 0 & \text{otherwise} \end{cases} \tag{24}$$

Having q_* makes choosing optimal actions even easier. With q_*, the agent does not even need to do a one-step-ahead search: for any state s, it can simply find any action that maximizes $q_*(s,a)$. The action-value function effectively caches the results of all one-step-ahead searches. It provides the optimal expected long-term return as a value immediately available for each state-action pair, enabling optimal action selection with no lookahead or model of the environment's dynamics required.

5.6 Solving the Bellman Optimality Equations

The Bellman optimality equations are a system of $|\mathcal{S}|$ nonlinear equations (for v_*) or $\sum_s |\mathcal{A}(s)|$ equations (for q_*). If we have the dynamics $p(s', r \mid s, a)$, then in principle we can solve this system for v_* or q_*.

In practice, this solution is rarely directly useful. It relies on three assumptions that are rarely true: (1) we accurately know the dynamics of the environment, (2) we have sufficient computational resources to complete the computation, and (3) the state space is small enough. For problems of practical interest, these assumptions typically do not hold.

Many RL methods can be viewed as approximately solving the Bellman optimality equation using actual experienced transitions in place of complete knowledge of expected transitions. The methods of dynamic programming make full use of the model but require enormous computational resources.

6 Dynamic Programming Methods

The term *dynamic programming* (DP) refers to a collection of algorithms that can be used to compute optimal policies given a perfect model of the environment as a Markov decision process. Classical DP algorithms are of limited utility in RL because they require a complete and accurate model of the environment, but they provide an essential foundation for understanding other methods.

6.1 Key Requirements and Ideas

DP algorithms require a complete model: all state transition probabilities $p(s', r \mid s, a)$ must be known. The key idea of DP (and of RL in general) is the use of value functions to organize and structure the search for good policies. Once we have found the optimal value function, v_* or q_*, the optimal policy can be obtained relatively easily.

6.2 Policy Evaluation (Prediction)

Policy evaluation, also called the *prediction problem*, is the task of computing the state-value function v_π for an arbitrary policy π. We use the Bellman equation for v_π as an update rule:

$$V(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)[r + \gamma V(s')] \tag{25}$$

This is an *iterative policy evaluation* algorithm. We initialize $V(s)$ arbitrarily (except that $V(\text{terminal}) = 0$), and then repeatedly apply the update to each state. Each iteration updates the value of every state to produce a new approximate value function. The sequence of approximate value functions $\{V_k\}$ converges to v_π as $k \to \infty$ under suitable conditions (guaranteed when $\gamma < 1$ or when eventual termination is guaranteed).

6.2.1 In-Place Algorithm

The algorithm can be implemented with two arrays, one for the old values and one for the new values, or with a single array that is updated in place. The in-place algorithm usually converges faster because it uses new data as soon as it is available. The order in which states are updated also affects the rate of convergence.

6.2.2 Gridworld Example

For a 4×4 gridworld with terminal states in opposite corners and reward of -1 on all transitions, iterative policy evaluation under the random policy shows values that become

increasingly negative as states are farther from the terminal states. The algorithm converges after several iterations.

6.3 Policy Improvement

Given the value function v_π for a policy π, we can decide whether we should change the policy to deterministically choose an action $a \neq \pi(s)$ in some state s. We know how good it is to follow the current policy from s (that's $v_\pi(s)$), but would it be better to first select action a and thereafter follow π?

We can answer this question by considering selecting a in s and thereafter following π. The value of this way of behaving is:

$$q_\pi(s,a) = \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_\pi(s')] \tag{26}$$

If this quantity is greater than $v_\pi(s)$, then it is better to select a once in s and thereafter follow π than to follow π all the time. We can generalize this idea to all states and all possible actions, selecting at each state the action that appears best according to $q_\pi(s,a)$:

$$\pi'(s) = \arg\max_a q_\pi(s,a) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_\pi(s')] \tag{27}$$

This is a greedy policy with respect to v_π. The policy π' is as good as or better than π. This is the *policy improvement theorem*.

Theorem 1 (Policy Improvement Theorem)**.** *Let π and π' be any pair of deterministic policies such that, for all $s \in \mathcal{S}$,*

$$q_\pi(s, \pi'(s)) \geq v_\pi(s) \tag{28}$$

Then the policy π' must be as good as or better than π. That is, it must obtain greater or equal expected return from all states $s \in \mathcal{S}$:

$$v_{\pi'}(s) \geq v_\pi(s) \tag{29}$$

If improvements stop—if the greedy policy π' is the same as the original policy π—then:

$$v_{\pi'}(s) = \max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_{\pi'}(s')] \tag{30}$$

This is the Bellman optimality equation, so $v_{\pi'} = v_*$ and both π and π' are optimal policies.

6.4 Policy Iteration

Once a policy π has been improved using v_π to yield a better policy π', we can compute $v_{\pi'}$ and improve it again to yield an even better π''. This process of iterating between policy evaluation and policy improvement is called *policy iteration*:

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \cdots \xrightarrow{I} \pi_* \xrightarrow{E} v_* \tag{31}$$

where $\xrightarrow{E}$ denotes policy evaluation and $\xrightarrow{I}$ denotes policy improvement.

Each policy is guaranteed to be a strict improvement over the previous one (unless it is already optimal). Because a finite MDP has only a finite number of deterministic policies, this process must converge to an optimal policy and optimal value function in a finite number of iterations.

6.4.1 Jack's Car Rental Example

A classic example is Jack's car rental problem. Jack manages two locations of a car rental company. Each day, customers arrive at each location to rent cars, and Jack can move cars between locations overnight. The problem is to determine how many cars to move to maximize expected profit, accounting for rental income, movement costs, and the stochastic arrivals and returns. Policy iteration efficiently finds the optimal policy.

6.5 Value Iteration

One drawback of policy iteration is that each iteration requires policy evaluation, which may itself be an iterative computation requiring multiple sweeps through the state set. We can truncate policy evaluation in several ways without losing convergence guarantees. An extreme case is to stop policy evaluation after just one sweep (one update per state). This algorithm is called *value iteration*.

Value iteration combines the policy improvement and truncated policy evaluation steps:

$$V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma V(s')] \tag{32}$$

This is identical to the Bellman optimality equation turned into an update rule. Value iteration effectively combines one sweep of policy evaluation and one sweep of policy improvement in each iteration.

Another way to understand value iteration is to compare the Bellman optimality equation with the Bellman equation for v_π. Notice that value iteration is simply policy evaluation using the Bellman optimality equation as the update rule. It converges to v_* under the same conditions that guarantee existence of v_*.

6.5.1 Gambler's Problem

A classic example is the gambler's problem. A gambler has the opportunity to make bets on the outcomes of a sequence of coin flips. If the coin comes up heads, the gambler wins as many dollars as staked on that flip; if tails, the stake is lost. The gambler wins by reaching a goal of \$100 and loses if all money is lost. The coin has probability p_h of coming up heads. The problem is to determine the optimal betting policy. Value iteration efficiently solves this problem.

6.6 Generalized Policy Iteration

Policy iteration consists of two simultaneous, interacting processes: one making the value function consistent with the current policy (policy evaluation), and the other making the

policy greedy with respect to the current value function (policy improvement). We use the term *generalized policy iteration* (GPI) to refer to the general idea of letting policy evaluation and policy improvement processes interact, independent of the granularity and other details.

Almost all RL methods are well described as GPI. They all have identifiable policies and value functions, with the policy always being improved with respect to the value function and the value function always being driven toward the value function for the policy.

6.6.1 Competing and Cooperating Processes

In GPI, evaluation and improvement processes can be viewed as both competing and cooperating. They compete in the sense that they pull in opposing directions. Making the policy greedy with respect to the value function typically makes the value function incorrect for the changed policy, and making the value function consistent with the policy typically causes the policy to no longer be greedy. In the long run, however, these two processes interact to find a single joint solution: the optimal value function and an optimal policy.

The evaluation and improvement processes in GPI can be viewed as both competing and cooperating. Making the greedy policy for the current value function is an improvement, but it also makes the value function incorrect. Making the value function consistent with the policy is evaluation, but the policy is no longer greedy. Yet both processes stabilize only when optimality is achieved.

6.7 Asynchronous Dynamic Programming

The DP methods described so far require systematic sweeps of the entire state set. *Asynchronous DP* algorithms update states in any order whatsoever, using whatever values of other states happen to be available. Some states may be updated several times before others are updated once. To converge correctly, however, asynchronous algorithms must continue to update all states: they cannot ignore any state indefinitely.

Asynchronous DP algorithms provide great flexibility in selecting states to update. We can try to take advantage of this flexibility by selecting states to which the agent is most likely to be in or by updating states with the largest estimated value change. We can even update the value of a single state on each step. If we select states in the order they are visited in real or simulated interaction with the environment, we can focus updates on the states that are most relevant.

6.8 Key Properties of Dynamic Programming

DP methods have several important properties:

Bootstrapping. DP methods update estimates based on other estimates. They use the value of successor states to update the value of the current state. This is a key property shared with many RL methods.

Computational Complexity. DP methods are guaranteed to find an optimal policy in polynomial time in the number of states and actions, even though the total number of deterministic policies is $|\mathcal{A}|^{|\mathcal{S}|}$. For large state spaces, however, even polynomial time can be prohibitive.

Curse of Dimensionality. The number of states often grows exponentially with the number of state variables. This is known as Bellman's *curse of dimensionality*. For example, a system with n binary state variables has 2^n possible states. This exponential growth makes even polynomial-time DP methods impractical for large problems.

Despite these limitations, DP provides an essential foundation for understanding RL methods. The asynchronous DP methods bridge the gap to RL: they can be executed during real interaction with the environment, updating states as they are encountered.

7 Summary and Quiz Preparation

For Quiz 1, the most critical concepts to understand are:

Chapter 1 Fundamentals:

- RL learns from interaction to maximize cumulative reward
- Distinguished by trial-and-error search and delayed reward
- Four elements: policy (behavior), reward (immediate goal), value function (long-term goal), model (optional)
- Exploration vs. exploitation tradeoff
- Temporal-difference update: $V(S_t) \leftarrow V(S_t) + \alpha[V(S_{t+1}) - V(S_t)]$

Chapter 3 MDPs:

- Dynamics: $p(s', r \mid s, a)$ completely characterizes environment
- Markov property: state captures all relevant history
- Return: $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = R_{t+1} + \gamma G_{t+1}$
- State-value: $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$
- Action-value: $q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$
- Bellman equation: $v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_\pi(s')]$
- Optimal value: $v_*(s) = \max_\pi v_\pi(s)$
- Bellman optimality: $v_*(s) = \max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_*(s')]$

Chapter 4 Dynamic Programming:

- Policy evaluation: iterative application of Bellman equation as update
- Policy improvement: greedy policy construction from value function
- Policy improvement theorem: greedy policy is at least as good
- Policy iteration: alternate evaluation and improvement until convergence

- Value iteration: combine evaluation and improvement in single sweep
- GPI: general framework where evaluation and improvement interact
- Requires complete model, polynomial time, suffers from curse of dimensionality
- Bootstrapping: updating estimates based on other estimates

Understanding these concepts, equations, and algorithms is essential for success on the quiz.